



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Centre for Professional and
Continuing Education

(SCTP) Advanced
Professional Certificate
Data Science and AI



3.7 Neural Networks & Deep Learning

Module Overview

3.1 Introduction to Machine Learning

3.2 Probability and Statistics for Machine Learning

3.3 Supervised Learning

3.4 Supervised Learning - Advanced

3.5 Unsupervised Learning

3.6 Time Series Data & Forecasting

3.7 Neural Network & Deep Learning

3.8 Computer Vision (CV)

3.9 Natural Language Processing (NLP)

3.10 Transformers, Attention & GenAI

L07

Neural Networks

Sarah's week of building her first neural network

Learning Objectives

- **Neural Networks:** Understand how multi-layered systems process and learn from complex data.
- **Activation Functions:** Learn how adding non-linearity helps the model capture complex patterns.
- **Forward Propagation:** Grasp how data flows through layers to generate a prediction.
- **Backpropagation:** Learn how the network calculates prediction errors using the chain rule.
- **Gradient Descent:** Understand how the model systematically tweaks weights to minimize error.
- **Optimizers:** Discover how algorithms dynamically adjust learning rates for faster training.

Where We Left Off

From time series to sequential behaviour

Marcus, end of L06:

"If we had DETAILED CUSTOMER BEHAVIOUR data — every click, every page view, every cart event —
could you predict whether they'd complete checkout WHILE they're shopping?"

What's different this week



Neural networks for the first time

MLP with hidden layers + non-linear activations



PyTorch as the framework

The 5-line training loop you'll use forever



Goal is INTUITION, not engineering

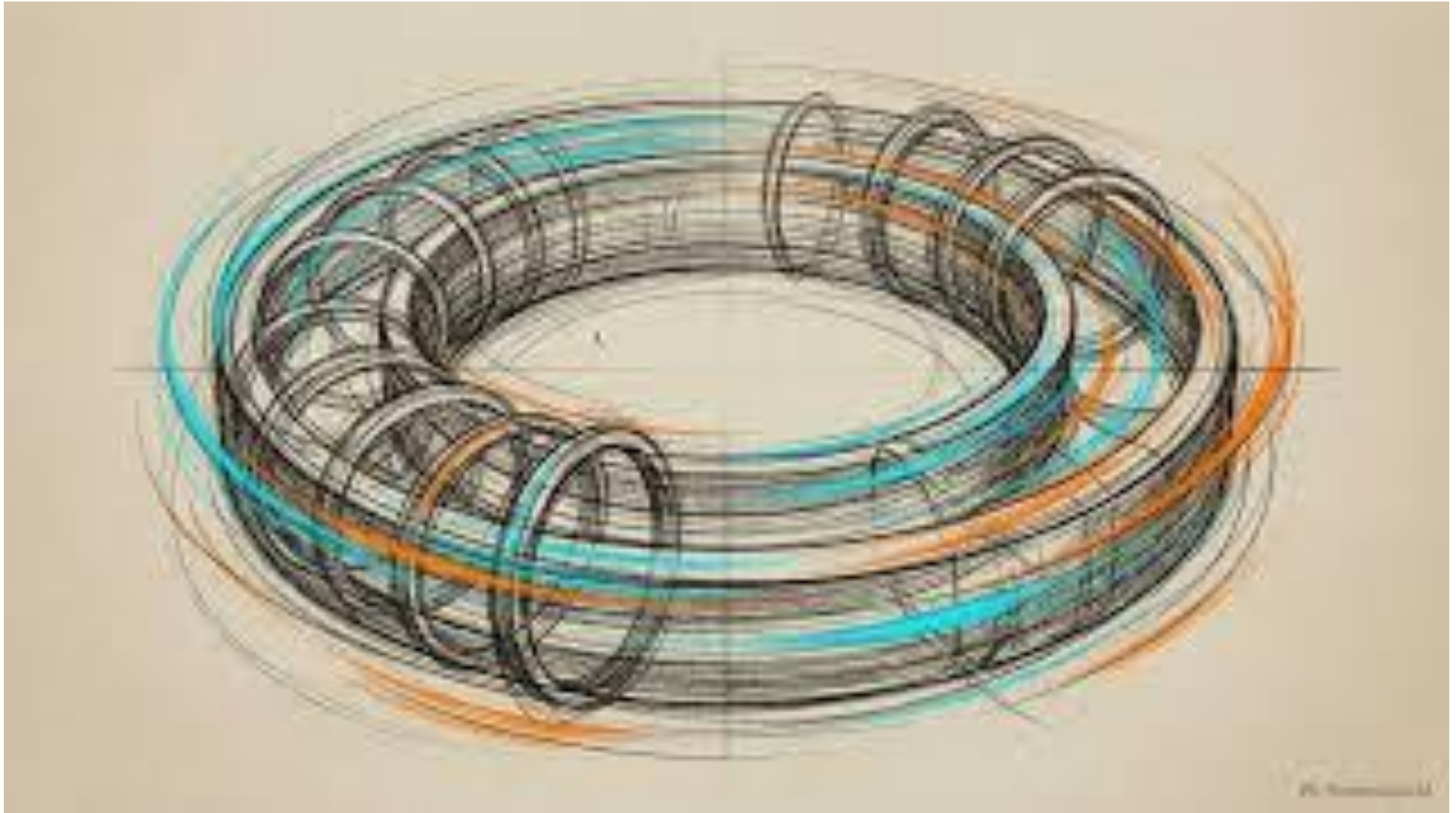
L08-L10 are where deep learning beats classical



Same compare-to-baselines mindset

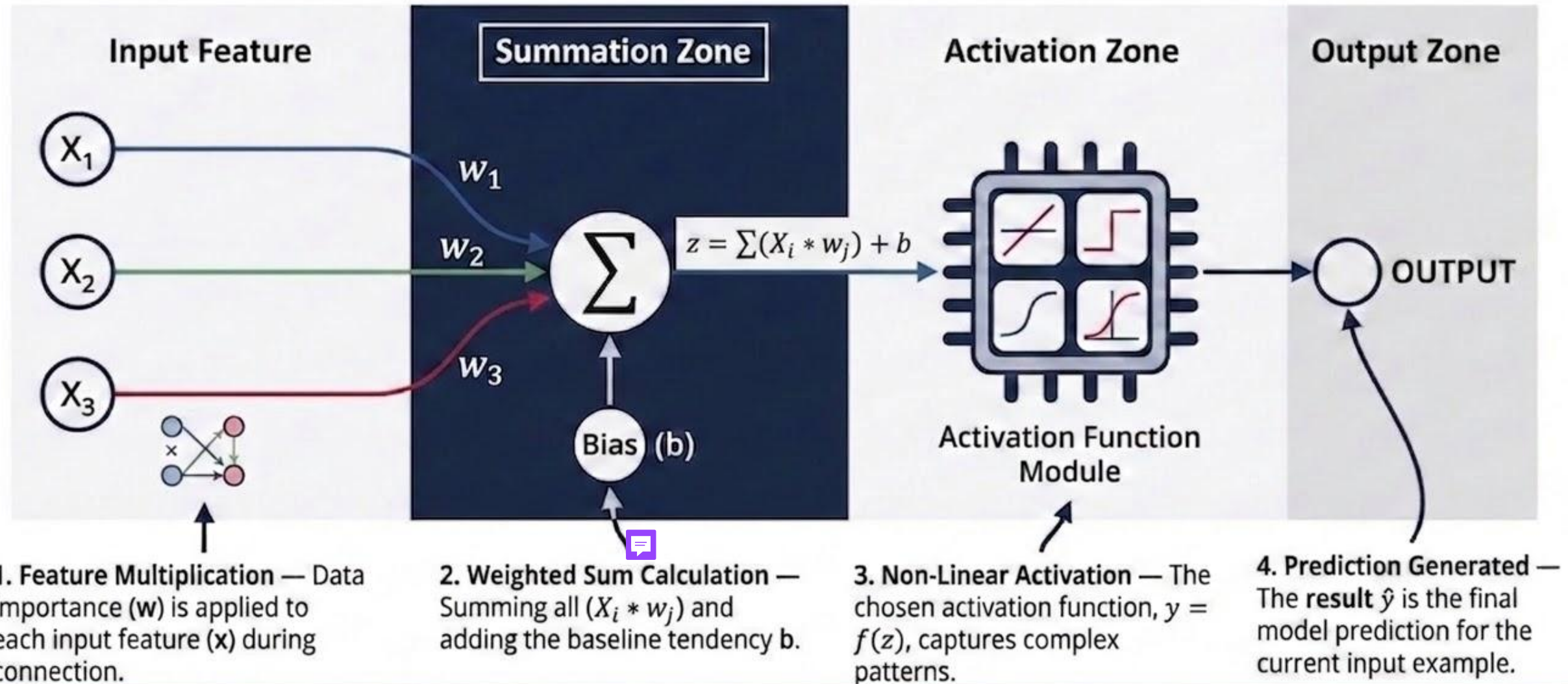
MLP vs LR vs gradient boosting on the same data

Neural Network - At a glance



SINGLE PERCEPTRON FORWARD PASS

From Inputs to Prediction: A Step-by-Step Visualization



Weights & Bias

Weights (w) – The Importance Finder

Weights determine how much influence an input feature has on the final output.

- If an input is highly important, its weight will be large.
- If an input is irrelevant, its weight will be close to zero.

Bias (b) – The Baseline Tendency

Bias represents the default setting of a neuron before considering any inputs. It allows you to shift your activation function up or down to better fit the data.

- Even if all your inputs (x) are zero, the neuron can still output a value because of the bias.

$$\text{Output} = (\text{Input} \times \text{Weight}) + \text{Bias}$$

The Perceptron — One Neuron

The 1958 building block

Formula

$$\text{output} = \text{activation}(w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b)$$

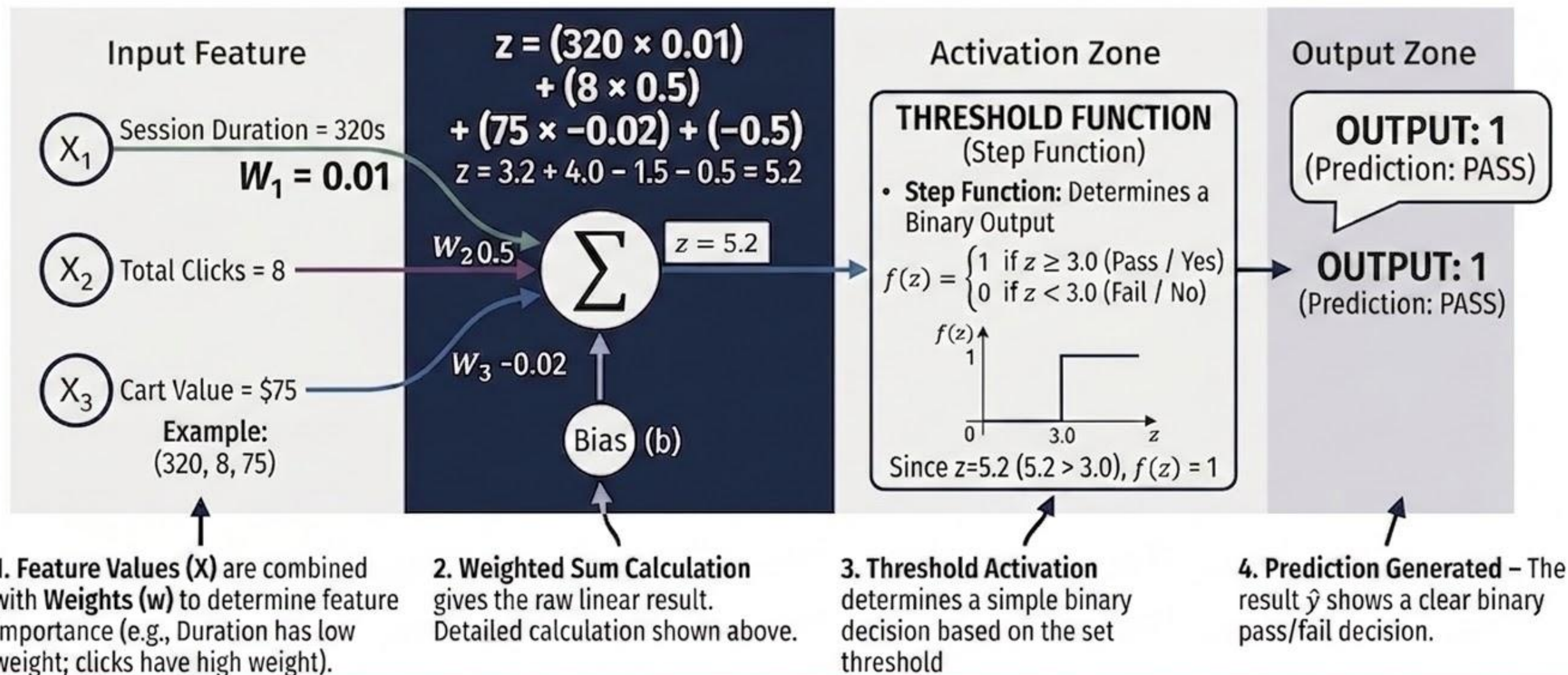
Activation function determines the model type

- ▶ **Linear (no activation)** = Linear regression
- ▶ **Threshold (step function)** = 1958 perceptron, binary output
- ▶ **Sigmoid** = Logistic regression (binary classification)
- ▶ **Softmax** = Multinomial logistic regression (multiclass)

 **A single perceptron can only learn LINEARLY SEPARABLE patterns. Fails on XOR.**

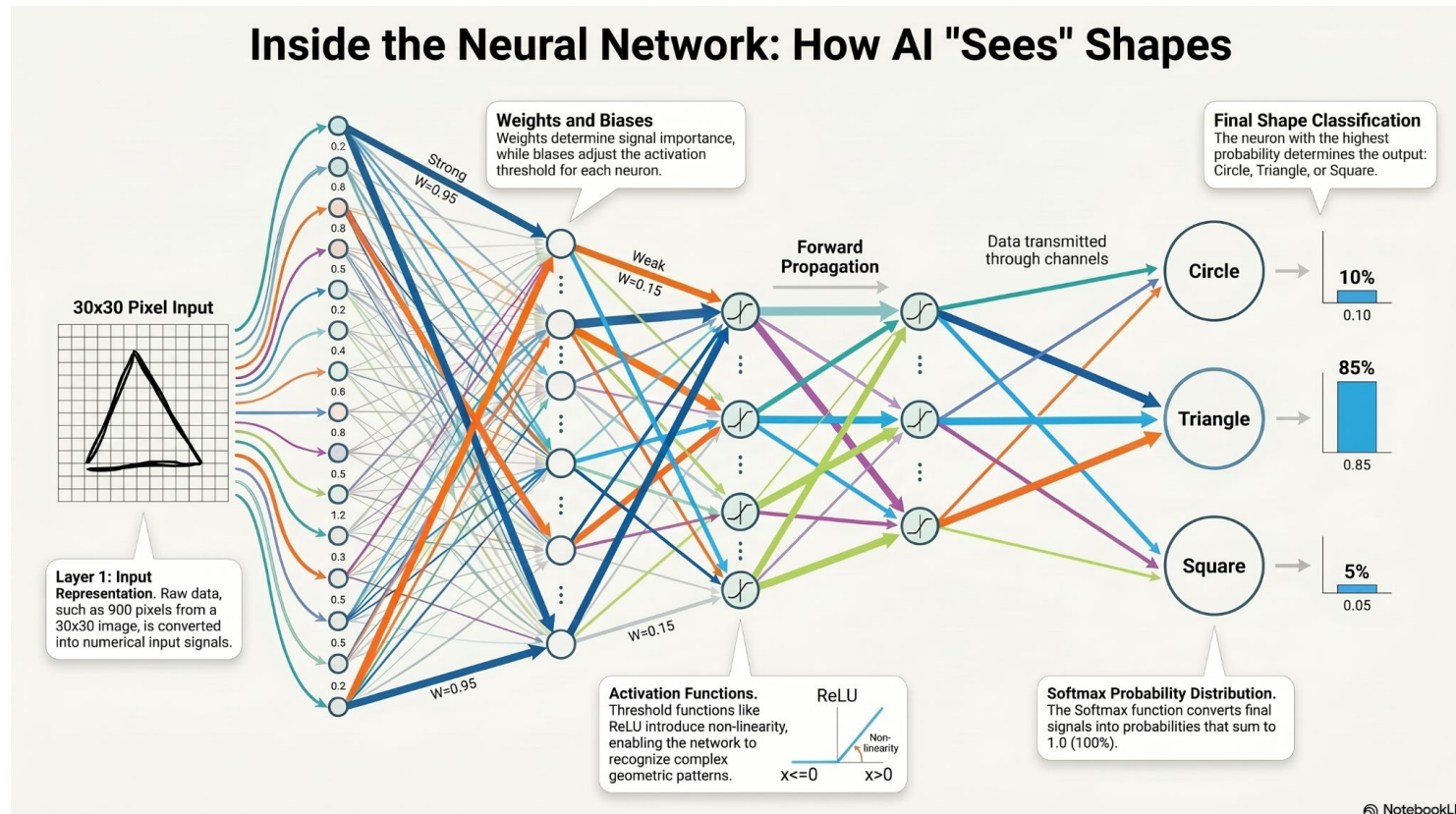
SINGLE PERCEPTRON FORWARD PASS: WORKED EXAMPLE

From Sample Inputs to a Threshold Prediction



Multi-Layer Perceptron (MLP)

Stack perceptrons + non-linear activations between layers



A single hidden layer with enough neurons + non-linear activations can approximate ANY continuous function arbitrarily well.

In practice: 2-3 hidden layers (a 'deep' network) trains better than one very wide layer with the same total parameters.

Neural Network in a nutshell

Three things, then we're done with the mystique

1

A FUNCTION

A flexible, non-linear function from inputs to outputs.
Learned automatically — you specify the architecture, not the function.

► **Forward pass: input → output**

2

A LOSS

A measure of how wrong the function is on training data.
For classification: cross-entropy. For regression: MSE.

► **Loss tells gradient descent which way to walk**

3

A LEARNING ALGORITHM

Gradient descent + backpropagation.
Walk downhill on the loss surface, one small step at a time.

► **Updates: $\text{weights} -= \text{lr} \times \text{gradient_of_loss}$**

The Gradient Descent



The Gradient Descent Update Rule

One line. Everything else is engineering.

$$\text{weights_new} = \text{weights_old} - \text{learning_rate} \times \text{gradient}$$

The learning rate

Too high

Overshoot the minimum, oscillate, diverge

Too low

Tiny steps, slow convergence, may never finish

Just right

Smooth, fast convergence

Adam optimiser adapts lr per parameter

THE Training Loop

Memorise these 5 lines — used everywhere in deep learning

```
for epoch in range(n_epochs):  
    for X_batch, y_batch in train_loader:           # iterate over mini-batches  
        optimizer.zero_grad()                     # 1. clear old gradients  
        predictions = model(X_batch)                # 2. forward pass  
        loss = criterion(predictions, y_batch)       # 3. compute loss  
        loss.backward()                             # 4. backward pass (autograd)  
        optimizer.step()                           # 5. update parameters
```

The pattern transfers everywhere

L08 (CNNs on images): same 5 lines, different model.

L09 (NLP): same 5 lines, different model.

L10 (transformers): same 5 lines, different model.

You don't derive backprop by hand. PyTorch (and TensorFlow, JAX, etc.) do it automatically via AUTOGRAD.

PyTorch — Four Objects

Tensor · nn.Module · DataLoader · Optimizer

Tensor

Multi-dim array with autograd

Replaces NumPy during training; tracks gradients automatically

nn.Module

The base class for any model

You subclass it, define layers + forward()

DataLoader

Wraps data for batched iteration

Used with TensorDataset for the typical workflow

Optimizer

Updates parameters from gradients

torch.optim.Adam, SGD, etc. — separate from the model

MLP vs Gradient Boosting — When To Use Which

On Sarah's tabular task: they're tied

MLP shines

- ✓ Image / text / audio data
- ✓ Very large datasets (>10M rows)
- ✓ Need to ensemble with other neural nets
- ✓ Sparse high-dim features
- ✓ When transfer learning available

Gradient Boosting shines

- ✓ Tabular data of moderate size
- ✓ Need feature importance for interpretability
- ✓ Limited training data
- ✓ Fast training preferred
- ✓ Few hyperparameters to tune

When To Reach For What

Quick reference for the L01-L07 toolkit

Tool	Best for	Strength	Weakness
Logistic regression	Linear binary problems	Interpretable	Linear only
Gradient Boosting	Tabular structured data	SOTA on tabular	Slow on huge data
MLP	Tabular foundation	Universal approximator	Many hyperparams
PyTorch (L07-L10)	Any custom neural architecture	Total flexibility	More code than sklearn

Friday — Sarah's Recommendation

What lands on Marcus's desk

Logistic Regression

AUC ~ 0.76

Top score, cheapest, most interpretable → SHIP THIS

Gradient Boosting

AUC ~ 0.75

Surprisingly the lowest of the four here

PyTorch MLP

AUC ~ 0.76

Foundation for L08-L10; not the production choice here

Marcus: 'Excellent. Next — automatically tag product PHOTOS as dress, jeans, jacket. Can you?'

Bridge to Lesson 8

Marcus, after the Friday presentation:

"Great. Now — we're launching an Instagram product feed. Can you automatically tag product PHOTOS so they categorise as 'dress', 'jeans', 'jacket', 'shoes', 'accessories', etc.?"

Lesson 8 — Computer Vision

CNN architecture · transfer learning · data augmentation

Same PyTorch training loop. New model architecture. Real images.

Before L08 — finish the L07 assignment (loan default + housing regression).

Assignment Preview

Two PyTorch exercises

Part A — Loan Default

- Binary classification
- 10,000 synthetic loan records
- 9 features
- Build the MLP class
- Train with Adam + early stopping
- Report test AUC

Part B — Housing Regression

- Regression (not classification)
- 8,000 synthetic housing rows
- MSELoss instead of BCE
- Same training loop pattern
- Report test MAE in \$100k units

Three Things to Remember

Beyond the algorithms

1

Neural networks are FUNCTIONS, not magic

They map inputs to outputs through stacked weighted sums + non-linear activations. Training is gradient descent on the loss surface.

2

The PyTorch training loop is THE pattern

5 lines that work for every deep-learning model you'll meet: `zero_grad`, `forward`, `loss`, `backward`, `step`. Memorise it.

3

On tabular data, the algorithm rarely matters

Neural networks shine on data with rich hierarchical structure (images, text). On tabular, they're competitive but rarely transformative.

MLP Architecture Cheatsheet

How many layers, how many neurons?

Tabular, ~10 features

1-2 hidden layers, 16-64 neurons

Sarah's session data falls here

Tabular, 100+ features

2-3 hidden layers, 64-256 neurons

Bigger but still modest

Images (will become CNNs)

Not MLP — use CNN (L08)

Convolutional inductive bias

**Text (will become
transformers)**

Not MLP — use transformer (L10)

Attention is the right tool

Sparse / very-high-dim

Wide + shallow (1 layer, many neurons)

Embeddings + 1 hidden layer

What's Coming — L08-L10

Specialising neural networks to specific data types

L08

Computer Vision

CNN

Convolutional layers exploit spatial structure in images.
Transfer learning from pre-trained models (ImageNet, CLIP).

L09

NLP

Word embeddings +
CNN/RNN

Token sequences. Word2Vec, GloVe, simple RNN classifiers.

L10

Transformers + GenAI

Attention

The architecture behind BERT, GPT, ChatGPT, Claude.
Fine-tuning + practical GenAI.

See You in L08

Computer Vision — CNNs, transfer learning, data augmentation

Before then: finish the L07 assignment (loan default + housing regression)

Or dig deeper with NumPy-from-scratch + weight init + Adam variants